

Query Processing for Heterogeneous Relational Data Integration

Jinpeng Wang, Yafei Zhang, Jianjiang Lu, Zhuang Miao, Bo Zhou
*Institute of Command Automation, PLA University of Science and Technology,
Nanjing 210007, China*
E-mail: wangjinpeng1982@gmail.com

Abstract—The emergence of the Semantic Web has given rise to researches on semantic integration of heterogeneous relational databases using ontology. Semantic-based heterogeneous relational data integration is recently attracting many diverse attentions, since most of the information is still stored in relational databases. This paper focuses on the query processing problem in semantic-based heterogeneous relational databases integration systems. The semantic of query rewriting is defined and algorithms for query decomposing, rewriting and transforming are presented. SPARQL queries over global schema are rewritten into local SQL queries that can be executed on heterogeneous relational databases. The experiment results show that the approach we proposed has nice performance and scalability.

Keywords– Data Integration; Relational Database; SPARQL; Query Processing.

I. INTRODUCTION

Integrating and querying data from heterogeneous sources is a hot research topic in database [1]. The goal of data integration is to provide user a uniform access to multiple heterogeneous data sources [2]. Considering the vast amount of data stored in heterogeneous relational databases, need for integrating all of these data efficiently is exigent. Traditional data integration systems, such as multi-database system or federal database system, are based on relational model. Due to the lack of semantic description capabilities, traditional data integration systems cannot resolve the semantic heterogeneity of data.

The burgeoning Semantic Web has provided several new approaches for data integration [3]. One of the most important visions of the Semantic Web research is ontology-based integration of heterogeneous relational database, which has recently received special attention in the database community [4, 5]. However, several important problems remain to be investigated. One of the primary obstacles in building a semantic-based data integration system is how to convert a global query into correct SQL query plans for accessing the relational databases. Systems like Virtuoso [6], D2RQ [7] and Squirrel RDF [8] rewrite SPARQL queries to SQL. The MySQL SPASQL [9] module compiles SPARQL queries directly into an evaluation structure to be executed by the relational engine.

However these research works only focused on accessing conventional relational databases using SPARQL, which didn't take the integration into account. A few early works study integration of relational databases using ontology [10–12]. All the three papers construct ontology from relational databases schema. This kind of method cannot fully capture the semantic behind the relational schema, only reflecting the structure of database, which makes it hard to integrate heterogeneous relational databases.

This paper focuses on the query processing problem for ontology-based relational data integration. In our proposal, RDF ontology is used as mediated schema for the explicit description of the data source semantics, providing a shared vocabulary for the specification of the semantics. The semantic of query rewriting is further discussed and a query rewriting algorithm is presented to reformulate a SPARQL query into SQL queries, so that SPARQL can access heterogeneous relational databases without converting the data into physical triples. We base our discussion on our previous work [13].

The remainder of this paper is structured as follows: In Section II defines the semantic for SPARQL queries and query rewriting. Section III discusses our query processing algorithms in detail. Algorithms for query decomposing, rewriting and transforming are presented. Section IV analyses the computational complexity of query rewriting algorithm and verifies its performance by experiment. Then the semantic equivalence of the query rewriting further discussed. Section V concludes the paper.

II. SEMANTICS OF SPARQL QUERIES

RDF describes things by making statements about an entity's properties. RDF statements are triples consisting of a subject (the resource being described), a predicate (the property) and an object (the property value). The values of a statement are URI references. This simple model of assertions leads to a network of information resources, interrelated by properties which establish relations between resources and property values. Thus, one can intuitively understand a collection of information resources and RDF statements describing them as a graph. To emphasize this characteristic, the term RDF Graph is defined as a set of RDF triples; hence, any collection of RDF data is an RDF Graph [14].

Definition 1 (RDF Terms, Triples, and Variables)

Assume there are pair wise disjoint infinite sets I , B , and L (IRIs, Blank nodes, and literals). A tuple $(s, p, o) \in (I \cup B) \times I \times (I \cup B \cup L)$ is called an RDF triple. In this tuple, s is the subject, p the predicate and o the object. We denote the union $I \cup B \cup L$ by T (RDF terms). Assume additionally the existence of an infinite set V of variables disjoint from the above sets.

Definition 2 (RDF Graph) An RDF graph is a set of RDF triples.

The problem of query processing considers how to reformulation a query over the RDF ontology to the underlying relational database. A survey and analysis on different algorithms to solve the problem is given in [15]. In this paper, we propose query processing algorithms over RDF Graph model.

Pérez introduce the formal semantics of SPARQL in [16]. The SPARQL query language is based on matching graph patterns. Graph patterns contain triple patterns that are like RDF triples, but with the option of query variables in place of RDF terms in the subject, predicate or object positions. Combining triple patterns gives a basic graph pattern, more complex graph patterns can be formed by combining smaller patterns.

The triple patterns of a SPARQL query over domain ontology can be considered as a sub-graph of the RDF Graph.

Definition 3 (Query Sub-graph) Given a SPARQL query Q , its Query Sub-graph G_Q is a set of RDF triples that satisfy the query.

Contents of a relational database can be considered as a sub-graph of RDF Graph.

Definition 4 (Table Sub-graph) Given a relational table P , its Table Sub-graph G_P is a set of RDF triples that satisfy the queries in source description of the relational table.

Given a SPARQL query over domain ontology, query rewriting algorithm finds proper relational tables whose sub-graph cover the SPARQL query, generates the query plans and unit the results together. The semantic of query rewriting is defined below:

Definition 5 (Query Rewriting) Given a SPARQL query Q over domain ontology, a query rewriting answer is a set of relational tables whose sub-graph cover the SPARQL query. The results of all possible query rewriting answers of Q make up the final result of Q .

III. QUERY PROCESSING ALGORITHMS

In a semantic-based heterogeneous data integration system, users are faced with a global model, putting forward queries based on it. The data integration system decompose global queries into sub-queries according to their content, rewrite the sub-queries to consist with the semantic of underlying relational databases, and then transform the rewriting result to SQL queries, so that the queries can be executed on relational databases. Query

processing includes query decomposing, rewriting and transforming. Algorithms for these three problems are presented as follow.

A. Query Decomposing

```

decompose ( $Q$ )
Input: global query  $Q = \text{SELECT } vars_Q \text{ WHERE } gp$ 
Output: sub-query set  $\{Q_1, Q_2, \dots, Q_n\}$ ,  $Q_i = \text{SELECT } vars_i \text{ WHERE } gp_i$ 

 $R = \emptyset$  //Initialize the returned result set
switch ( $gp$ )
  case  $gp = bgp$  //If the query graph pattern is basic graph pattern
    add  $Q$  to  $R$ 
    return  $R$  //return  $R$ 
  //If input group graph pattern, alternative graph pattern or optional graph pattern
  case  $gp = gp_a \text{ AND } gp_b$ :
  case  $gp = gp_a \text{ UNION } gp_b$ :
  case  $gp = gp_a \text{ OPTION } gp_b$ :
    //Construct sub-queries
    construct  $Q_a = \text{SELECT } vars_a \text{ WHERE } gp_a$ 
     $vars_a = (var(gp_a) \cap vars_Q) \cup (var(gp_a) \cap var(gp_b))$ 
    construct  $Q_b = \text{SELECT } vars_b \text{ WHERE } gp_b$ 
     $vars_b = (var(gp_b) \cap vars_Q) \cup (var(gp_a) \cap var(gp_b))$ 
     $R_1 = \text{decompose}(Q_a)$ ; //recursive call the query decomposing function
     $R_2 = \text{decompose}(Q_b)$ ;
    return  $R_1 \cup R_2$  //return the result
  case  $gp = gp_a \text{ FILTER } expr$ : //If the query graph pattern contains FILTER function
    //Construct sub-queries without FILTER
    construct  $Q_a = \text{SELECT } vars \text{ WHERE } gp_a$ 
     $R = \text{decompose}(Q_a)$ ; //call the query decomposing function
    for each  $Q_i = \text{SELECT } vars_i \text{ WHERE } gp_i$  in  $R$ ,
       $Q_i = \text{SELECT } vars_i \text{ WHERE } gp_i \text{ FILTER } expr$ 
    return  $R$  //return the result

```

Figure 1 query decomposing algorithm

B. Query Rewriting

As implied by the previous section, a semantically correct query plan amounts to finding proper relational tables whose sub-graph cover the SPARQL query. We present the query rewriting algorithm to generate semantically correct and executable query plans under the RDF Graph model. Our algorithm proceeds in two stages. In the first stage, we find all relative tables and decompose them to Minimal Connectable Units (MCU) according to source descriptions. In the second stage we join MCUs to produce query plans that are semantically correct and executable.

a) Generating the MCUs

A MCU is a subset of RDF triples in source description of a relational table that can be joined with other MCUs and executed on heterogeneous databases. Intuitively, a MCU

represents a fragment of semantic mapping from the query to the rewriting of the query.

Formally, we define MCUs as follows.

Definition 6 (Minimal Connectable Units) Given a SPARQL query Q , a relational table P and its Table Sub-graph G_P , a MCU m for P is a tuple of the form (Y, μ') where:

- Y is a subset of triple patterns in G_P .
- μ' is a partial mapping from variables appeared in G_P to corresponding columns in P .

```

findMCUs( $Q, G_P$ )
/*  $Q$  is a SPARQL query,  $G_P$  is the Table Sub-graph of a relational
table  $P$  */
Initialize  $M = \Phi$ ;
For each triple pattern  $t$  in  $Q$ , do
  for each triple pattern  $t'$  in  $G_P$ , do
    if exist a mapping  $\tau$  that map  $t$  to  $t'$ , then
      find the minimal subset (denoted by  $Y$ ) of triple patterns of
 $G_P$  that is connectable.
      find the subset (denoted by  $\mu'$ ) of  $\mu$  that relative to  $Y$ .
       $M = M \cup \langle Y, \mu' \rangle$ ;
    end for
  end for
return  $M$ ;

```

Figure 2 find the MCUs

The algorithm for creating the MCDs is shown in Figure 2. We say a set of triple patterns Y connectable if the following conditions hold: (1) Each return variable of Q is also return variable in Y . (2) If x is not a return variable in Q , then for every triple pattern t that includes x , $t \in Y$.

b) Joining the MCUs

In this phase we consider combinations of MCUs, and for each valid combination we create a conjunctive rewriting of the query. The final result is a union of conjunctive queries. Given a set of MCUs, the actual rewriting is constructed as shown in Figure 3.

```

joinMCUs( $Q, M, A$ )
/*  $Q$  is a SPARQL query,  $M$  is a set of MCUs,  $M = \{m_1, \dots, m_n\}$ ,
where  $m_i = (Y_i, \mu_i)$ ,  $A$  is a set of rewritings */
Initialize  $A = \Phi$ ;
for each minimal subset  $\{m_1, \dots, m_k\}$  of  $M$  such that
   $Y_1 \cup Y_2 \cup \dots \cup Y_k$  cover all triple patterns of  $Q$ .
  Create the conjunctive rewriting  $Q'$  contain all relative tables
to  $\{m_1, \dots, m_k\}$ 
  Add  $Q'$  to  $A$ 
end for
return  $A$ 

```

Figure 3 join the MCUs

C. Query Transforming

After rewriting, the sub-queries are consistent with the semantic of underlying relational databases. But they still can not execute on relational databases. The rewriting result should transform into SQL queries for accessing the

relational databases. The query transforming algorithm is shown in Figure 4 and Figure 5.

```

queryTransformation( $gp$ )
input: SPARQL query  $Q = \text{SELECT } vars \text{ WHERE } gp$ 
output: SQL query plans
for each variable  $v_i$  in  $vars$ 
  add  $v_i$  in  $varlist$ 
 $SQL = \text{"SELECT DISTINCT"} + varlist +$ 
 $\text{"FROM"} + \text{graphPatternTranslation}(gp)$ 
return  $SQL$ 

```

Figure 4 Query transforming algorithm

```

graphPatternTranslation( $gp$ )
input: graph pattern  $gp$ 
output: SQL query
//According to different type of graph pattern switch( $gp$ )
let  $varlist_a = vars_a - vars_b$ 
     $varlist_b = vars_b - vars_a$ 
     $varlist_c = vars_a \cap vars_b$ 
//group graph pattern
case  $gp = gp_a \text{ AND } gp_b$ :
 $SQL = \text{"SELECT DISTINCT"} + varlist_a + varlist_b$ 
for each variable  $c$  in  $varlist_c$ 
 $SQL += \text{"COALESCE}(R_1.c, R_2.c) \text{ AS } c"$ 
 $SQL += \text{"FROM (graphPatternTranslation}(gp_a)) R_1 \text{ INNER JOIN}$ 
 $\text{(graphPatternTranslation}(gp_b)) R_2 \text{ ON}"$ 
for each variable  $c$  in  $varlist_c$ 
 $SQL += \text{"} R_1.c = R_2.c \text{ OR } R_1.c \text{ is NULL OR } R_2.c \text{ is NULL}"$ 
return  $SQL$ 
//alternative graph pattern
case  $gp = gp_a \text{ UNION } gp_b$ :
 $SQL = \text{"SELECT"} + varlist_a + varlist_b + varlist_c +$ 
 $\text{"FROM"} + \text{(graphPatternTranslation}(gp_a)) R_1 + \text{"LEFT JOIN"} +$ 
 $\text{(graphPatternTranslation}(gp_b)) R_2 + \text{"UNION"} +$ 
 $\text{(graphPatternTranslation}(gp_b)) R_2 + \text{"LEFT JOIN"} +$ 
 $\text{(graphPatternTranslation}(gp_a)) R_1$ 
return  $SQL$ 
//optional graph pattern
case  $gp = gp_a \text{ OPTION } gp_b$ :
 $SQL = \text{"SELECT DISTINCT"} + varlist_a + varlist_b$ 
for each variable  $c$  in  $varlist_c$ 
 $SQL += \text{"COALESCE}(R_1.c, R_2.c) \text{ AS } c"$ 
 $SQL += \text{"FROM (graphPatternTranslation}(gp_a)) R_1 \text{ LEFT JOIN}$ 
 $\text{(graphPatternTranslation}(gp_b)) R_2 \text{ ON}"$ 
for each variable  $c$  in  $varlist_c$ 
 $SQL += \text{"} R_1.c = R_2.c \text{ OR } R_1.c \text{ is NULL OR } R_2.c \text{ is NULL}"$ 
return  $SQL$ 
//graph pattern constraint
case  $gp = gp \text{ FILTER } expr$ :
 $SQL = \text{"SELECT * FROM"} + \text{graphPatternTranslation}(gp)$ 
 $\text{WHERE } expr$ 
return  $SQL$ 

```

Figure 5 Graph pattern transforming algorithm

IV. EVALUATION

The proposed algorithms in this paper have been fully implemented on a Pentium Dual 1.79GHz computer with the Java 1.6. In order to meet the requirement of practical applications, we have constructed 10 different relational

databases, containing 100 heterogeneous relational tables. We also construct 100 different global queries with length from 1 to 10 to simulate various user queries.

A. Computational Complexity

In query decomposing algorithm, given a SPARQL query, if it contains some nested operations, query decomposing algorithm will be called repeatedly. Let n stands for the number of triples in query pattern; and $k(1 < k < n-1)$ stands for the number of nested operations in query. Under the worst case, query decomposing algorithm can be finished in $O(n^2)$ time. Its time complexity is polynomial.

As for the query rewriting algorithm, given a set of data source $V = \{v_1, v_2, \dots, v_n\}$ and query $Q = \text{SELECT vars WHERE } gp, |gp| = m$, query rewriting algorithm needs to find all possible subset of V that can form a MCU, which requires $O(n^m)$ time. However, as the first step of query rewriting already rule out the nonrelevant data source, significantly cutting the unnecessary search branches, it can be expected that the actual operating efficiency of the query rewriting algorithm will be more satisfactory.

B. Experiment Results

In order to verify performance of query processing algorithms we have proposed, we design the following experiment. Figure 6 shows that the performance of query processing scalable in terms of length of query: the running time is almost exponential to the length of query. An interesting finding is that the running time is not sensitive to the length of query when number of data sources is less than 50. Figure 7 shows that the performance of query processing scalable in terms of number of data sources: the running time is almost exponential to the number of data sources. But the running time is not sensitive to the length of query when length of query is less than 8.

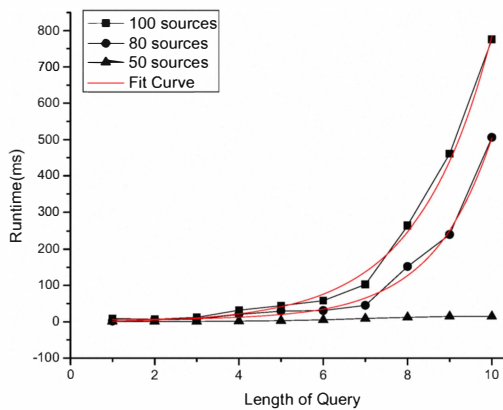


Figure 6 Performance test of query processing (1)

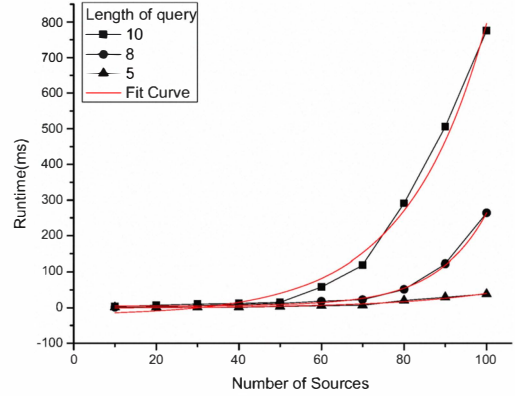


Figure 7 Performance test of query processing (2)

C. Discussions

The experimental results are consistent with the analysis of time complexity. When the length of query is less than 8, but lot of data sources (more than 80), the query processing algorithms can provide nice performance (within 500 milliseconds). This is also consistent with the practical circumstances in integration systems, where the query length is usually short.

V. CONCLUSIONS AND FUTURE WORK

The integration of data from multiple heterogeneous sources is an old and well-known research problem for the database and AI research communities. In this paper we considered the query processing problem in heterogeneous data integration systems. We use ontology as the mediated schema for integration. The semantic of query rewriting is defined. Algorithms for query decomposing, rewriting and transforming are presented, so that SPARQL queries over global schema can be rewritten into local SQL queries that can be executed on heterogeneous relational databases.

The approach and algorithms we provided in this paper can be extended for other data source (i.e. RDF, OWL, web forms, etc.); some of the complex semantic (i.e. CONSTURCT, ASK) are not discussed. We leave these problems for future work.

ACKNOWLEDGEMENTS

This paper is partially supported by the Weaponry Equipment Pre-Research Foundation of PLA Equipment Ministry (No. 9140A06050409JB8102) and Pre-Research Foundation of PLA University of Science and Technology (No. 2009JSJ11). The authors also gratefully acknowledge the helpful comments and suggestions of the anonymous reviewers.

REFERENCES

- [1] A. Y. Halevy, A. Rajaraman, and J. J. Ordille, "Data integration: the teenage years", Proceedings of the 32nd

- International Conference on Very Large Data Bases, 2006, pp. 9-16.
- [2] A. Y. Levy, A. Rajaraman, and J. J. Ordille, "Querying Heterogeneous Information Sources Using Source Descriptions," in Proceedings of the 22th International Conference on Very Large Data Bases, San Francisco, CA, USA, pp. 251-262, 1996.
 - [3] T. Berners-Lee, "Relational Databases on the Semantic Web", <http://www.w3.org/DesignIssues/RDB-RDF.html>, 1998.
 - [4] D. Dou and P. LePendu, "Ontology-based integration for relational databases", Proceedings of ACM Symposium on Applied computing, Dijon, France, 2006. pp. 461-466.
 - [5] H. Zhu and S. Madnick, "A Lightweight Ontology Approach to Scalable Interoperability", Very Large Data Bases Workshop on Ontology-based techniques or DataBases and Information Systems, Seoul, Korea, 2006, pp. 45-54.
 - [6] O. Erling and I. Mikhailov, "RDF Support in the Virtuoso DBMS," in Proceedings of the 1st Conference on Social Semantic Web, Leipzig, Germany, pp. 59-68, 2007.
 - [7] C. Bizer and A. Seaborne, "D2RQ – Treating Non-RDF Databases as Virtual RDF Graphs (Poster)," in 3rd International Semantic Web Conference, Hiroshima, Japan, 2004.
 - [8] G. Zhou, R. Hull, R. King, and J.-C. Franchitti, "Using object matching and materialization to integrate heterogeneous databases," In Proceedings. of the 3rd International Conference on Cooperative Information Systems (CoopIS'95), Vienna, Austria, pp. 4-18, 1995.
 - [9] E. Prudhommeaux, "SPASQL: SPARQL Support In MySQL," <http://xtech06.usefulinc.com/schedule/paper/156>, 2006.
 - [10] C. P. d. Laborda and S. Conrad, "Bringing Relational Data into the Semantic Web using SPARQL and Relational.OWL," in Proceedings of the 22nd International Conference on Data Engineering Workshops, Atlanta, GA, USA, p. 55, 2006.
 - [11] R. J. Bayardo, W. Bohrer, R. Brice, and et al., "Infosleuth: Agent-Based Semantic Integration of Information in Open and Dynamic Environments," SIGMOD Record vol. 26, pp. 195-206, 1997.
 - [12] D. Dou and P. LePendu, "Ontology-based Integration for Relational Databases," in Proceedings of the 2006 ACM symposium on Applied computing, Dijon, France, pp. 461-466, 2006.
 - [13] J. Wang, J. Lu, Y. Zhang, and et al., "Integrating heterogeneous data source using ontology", Journal of Software, 2009, Vol. 4, No. 8, pp. 843-850.
 - [14] J. Hayes, "A graph model for RDF," PhD dissertation, Darmstadt University of Technology, Germany, 2004.
 - [15] R. Pottinger and A. Halevy, "MiniCon: A Scalable Algorithm for Answering Queries Using Views," The VLDB Journal, vol. 10, pp. 182-198, 2001.
 - [16] J. Pérez, M. Arenas, and C. Gutierrez, "Semantics of SPARQL," Technical Report, Universidad de Chile TR/DCC-2006-17, 2006.